

Macro Scenario Engine

Motor de Análise de Cenários Macroeconômicos para o Mercado Brasileiro

1. Visão Geral do Sistema

O objetivo principal desta aplicação é fornecer análises macroeconômicas instantâneas, humanizadas e direcionadas para o mercado financeiro brasileiro (B3) a partir de entradas de texto livre fornecidas por usuários em um fluxo automatizado de atendimento.

Fluxo de Integração de Ponta a Ponta:

- O usuário interage com o assistente virtual via **Typebot**, informando seu nome e descrevendo um cenário econômico atual de interesse.
- O Typebot intercepta essa mensagem e dispara uma requisição estruturada HTTP do tipo POST para o back-end.
- A API construída em **FastAPI (Python)**, rodando sob infraestrutura Serverless na **Vercel**, valida e higieniza os dados recebidos.
- A API efetua uma chamada direta aos servidores de alta velocidade da **Groq Cloud**, utilizando o modelo de linguagem **Llama 3.3 70B Versatile**.
- O modelo contextualiza e gera insights estratégicos com base em engenharia de prompt focada no ecossistema financeiro brasileiro.
- A resposta é encapsulada e devolvida de forma síncrona ao Typebot, que renderiza as sugestões de forma legível e interativa ao usuário final.

2. Arquitetura Técnica e Tecnologias

A stack de componentes foi desenhada visando latência ultrabaixa, alta disponibilidade e custo zero de manutenção:

- **Interface (Front-end):** Plataforma Typebot.
- **Framework Web (Back-end):** FastAPI (Python 3.10+), proporcionando tipagem estática com Pydantic e rápida execução.
- **Camada Serverless:** Componente Mangum, atuando como adaptador ASGI para a execução nativa no ecossistema AWS/Vercel Lambda.
- **Provedor de Inteligência Artificial:** API Groq Cloud acoplada ao modelo de alta performance llama-3.3-70b-versatile.

- **Hospedagem e Infraestrutura:** Vercel Platform (Hobby Tier), garantindo escalabilidade automática sob demanda.

3. Código Fonte Homologado

Arquivo: `main.py`

Estrutura otimizada para o ecossistema Serverless, utilizando conexões diretas via biblioteca `requests` para contornar restrições de tempo limite (timeout) de gateway HTTP/2.

```

import os
import requests
from fastapi import FastAPI
from pydantic import BaseModel
from dotenv import load_dotenv
from mangum import Mangum

if os.path.exists("./.env"):
    load_dotenv(dotenv_path="./.env")

app = FastAPI(title="Macro Scenario Engine API")
handler = Mangum(app)

class CenarioInput(BaseModel):
    cenario: str

@app.post("/analisar")
def analisar_cenario(dados: CenarioInput):
    texto_recebido = dados.cenario
    api_key = os.environ.get("GROQ_API_KEY")

    if not api_key:
        return {
            "status": "erro",
            "mensagem": "Erro interno: Chave GROQ_API_KEY ausente na Vercel."
        }

    prompt = f"Você é um especialista em análise macroeconômica para o mercado financeiro brasileiro. Analise o cenário: {texto_recebido} e traga recomendações estruturadas dos setores da B3."

    url = "https://api.groq.com/openai/v1/chat/completions"
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": "llama-3.3-70b-versatile",
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0.7
    }

    try:
        response = requests.post(url, json=payload, headers=headers, timeout=8)
        if response.status_code == 200:
            dados_resposta = response.json()
            analise_real = dados_resposta["choices"][0]["message"]["content"]
        else:
            analise_real = f"Desculpe, a API do motor retornou erro código

```

```
{response.status_code}."
except Exception as e:
    analise_real = f"Erro de conexão com o motor de análise: {str(e)}"

return {
    "status": "sucesso",
    "mensagem": analise_real
}
```

Arquivo: `requirements.txt`

```
fastapi
pydantic
python-dotenv
mangum
requests
```

Arquivo: `vercel.json`

```
{
  "version": 2,
  "builds": [
    {
      "src": "main.py",
      "use": "@vercel/python"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "main.py"
    }
  ]
}
```

4. Integração com o Typebot (Webhook Configuration)

Configuração aplicada dentro do nó de requisição HTTP do fluxo do chatbot:

- **Método:** POST
- **URL:** <https://macro-scenario-engine.vercel.app/analisar>
- **Headers:** Content-Type: application/json
- **Body (JSON):**

```
{
  "cenario": "{{{variavel_cenario_usuario}}}"
}
```

5. Engenharia de Soluções e Resolução de Problemas (Engineering Notes)

Mitigação do Timeout de Rede: O uso do SDK oficial da Groq impunha requisições de rede incompatíveis com as travas de segurança da infraestrutura Serverless Hobby da Vercel, resultando em quedas intermitentes de conexão. A solução foi arquitetada utilizando requisições HTTP limpas via `requests` com controle de timeout rigoroso em 8 segundos.

Segurança e Isolamento: A credencial `GROQ_API_KEY` foi isolada totalmente do ambiente de versionamento `git`, sendo configurada em variáveis de ambiente nativas e criptografadas na Vercel, cumprindo os padrões de proteção de chaves de serviço.

Comportamento do Endpoint Raiz: A requisição direta via navegador à raiz do projeto retornará o JSON corporativo `{"detail": "Not Found"}`. Isso constitui um comportamento padrão intencional de APIs RESTful, visto que apenas o endpoint `/analisar` está exposto para o recebimento de cargas via método `POST`.

Documentação final técnica homologada para avaliação oficial do estudo de caso.